UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e INTERACCIÓN HUMANO COMPUTADORA

EJERCICIOS DE CLASE Nº 03

NOMBRE COMPLETO: Pérez Uribe José Alberto

Nº de Cuenta: 318143213

GRUPO DE LABORATORIO: 02

GRUPO DE TEORÍA: 07

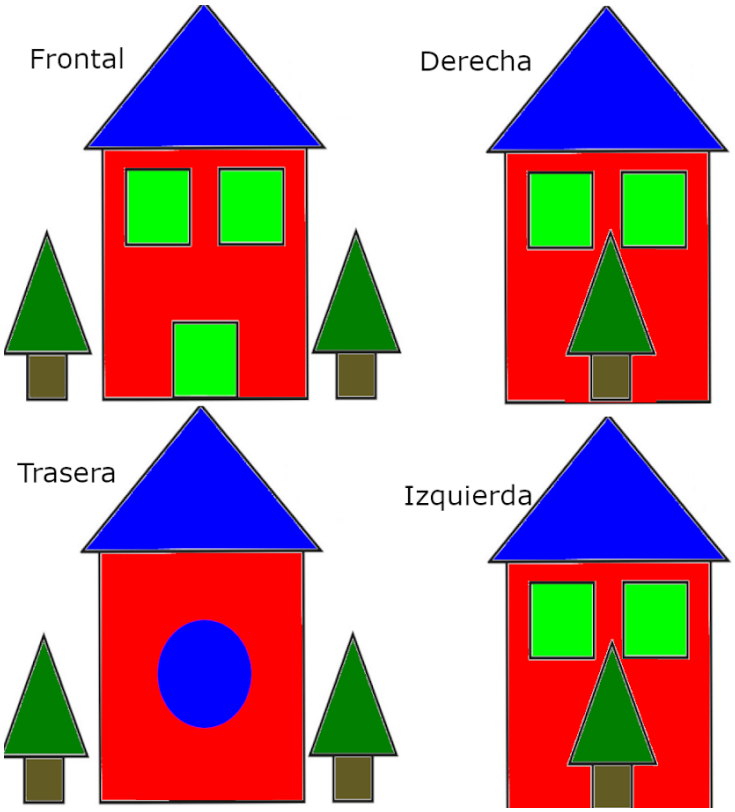
SEMESTRE 2026-2

FECHA DE ENTREGA LÍMITE: 03/03/26

CALIFICACIÓN: _____

Ejercicio

Desarrollar un modelo 3D basado en la composición de la práctica anterior mediante la instanciación de primitivas geométricas. El diseño debe integrar un plano de suelo (piso) y una estructura habitacional con las siguientes especificaciones: una ventana circular azul centrada en la fachada posterior, dos ventanas verdes en cada muro lateral (idénticas a las frontales) y un acceso único situado en la pared frontal.



El objetivo de este ejercicio fue la transición de un dibujo 2D realizado en prácticas anteriores a un entorno tridimensional, para lograrlo, se desarrolló una escena arquitectónica que incluye una casa detallada y un plano de suelo, aplicando conceptos de modelado geométrico, transformaciones lineales (traslación, rotación y escala) y el manejo de una cámara sintética con control de tiempo para una navegación fluida.

Durante el desarrollo de la práctica se identificó que algunas primitivas geométricas (principalmente la pirámide, el cono y el cilindro) presentaban deformaciones al renderizarse. Esto se debía a que en la función RenderMeshGeometry() del archivo Mesh.cpp se estaba utilizando el modo de dibujo GL_TRIANGLE_FAN.

```
glDrawElements(GL_TRIANGLE_FAN, indexCount, GL_UNSIGNED_INT, 0);
```

El modo GL_TRIANGLE_FAN conecta todos los vértices tomando el primero como punto central, formando un abanico de triángulos. Sin embargo, las primitivas geométricas implementadas en esta práctica fueron construidas mediante índices agrupados de tres en tres, es decir, cada grupo de tres índices define un triángulo independiente.

Debido a esta diferencia, OpenGL interpretaba incorrectamente la conectividad de los vértices, generando triángulos cruzados o superficies deformadas.

Para corregir este comportamiento se modificó la línea anterior por:

```
glDrawElements(GL_TRIANGLES, indexCount, GL_UNSIGNED_INT, 0);
```

Con el modo GL_TRIANGLES, cada conjunto de tres índices se interpreta como un triángulo independiente, lo cual coincide con la forma en que fueron construidos los índices de las primitivas geométricas.

Este cambio implicó que las funciones de generación de geometría (CrearCilindro, CrearCono y CrearPiramideCuadrangular) tuvieran que definir explícitamente los triángulos que componen cada superficie mediante arreglos de índices. De esta forma se garantiza que cada cara del objeto sea dibujada correctamente.

Gracias a esta modificación fue posible renderizar adecuadamente todas las primitivas utilizadas en la escena final, evitando deformaciones en el techo de la casa y en los árboles generados con cilindros y conos.

Una vez definidas las clases base proporcionadas por el profesor (Mesh, Shader, Camera, Window y Sphere), se implementaron funciones adicionales para generar las primitivas geométricas utilizadas en la escena. Estas funciones se encargan de construir los vértices y los índices necesarios para representar cada figura mediante triángulos.

Las primitivas generadas fueron almacenadas en un contenedor vector<Mesh*> meshList, lo que permite reutilizar la misma geometría varias veces dentro de la escena mediante transformaciones de modelo.

Cada figura se implementó mediante una función que genera su geometría a partir de vértices e índices. Estos datos se envían posteriormente al objeto Mesh, que se encarga de almacenarlos en los buffers de OpenGL (VBO, IBO y VAO).

En esta práctica las primitivas se construyen generando vértices (coordenadas 3D) y índices (qué 3 vértices forman cada triángulo). Los datos se guardan en un objeto Mesh y se almacenan en meshList para reutilizarlos.

Se usan dos formas de crear mallas:

- CreateMesh(vertices, indices, ...) para arreglos estáticos (cubo).
- CreateMeshGeometry(verticesVector, indicesVector, ...) para geometría generada dinámicamente (cilindro, cono, pirámide).

Cubo

El cubo se define con 8 vértices (24 valores) y 36 índices (12 triángulos = 6 caras).

```
void CrearCubo() {
    unsigned int cubo_indices[] = {
        0, 1, 2, 2, 3, 0, 1, 5, 6, 6, 2, 1,
        7, 6, 5, 5, 4, 7, 4, 0, 3, 3, 7, 4,
        4, 5, 1, 1, 0, 4, 3, 2, 6, 6, 7, 3
    };
    GLfloat cubo_vertices[] = {
        -0.5f, -0.5f,  0.5f,  0.5f, -0.5f,  0.5f,  0.5f,  0.5f,  0.5f, -0.5f,  0.5f,  0.5f,
        -0.5f, -0.5f, -0.5f,  0.5f, -0.5f, -0.5f,  0.5f,  0.5f, -0.5f, -0.5f,  0.5f, -0.5f
    };
    Mesh* cubo = new Mesh();
    cubo->CreateMesh(cubo_vertices, cubo_indices, 24, 36);
    meshList.push_back(cubo); // meshList[0]
}
```

Este cubo se utiliza como base para el piso, el cuerpo de la casa, las ventanas y la puerta.

Cilindro

se genera una circunferencia con resolución res, usando:

$x = R\cos(\theta)$

$z = R\sin(\theta)$

donde $\theta = n \cdot 2\pi/res$

Para cada n se crean dos vértices:

- uno abajo (y = -0.5)
- uno arriba (y = 0.5)

Después se generan los vértices de las circunferencias inferior y superior, los cuales serán utilizados para construir la base y la tapa del cilindro. También se agregan dos vértices centrales que representan el centro de la base y el centro de la tapa.

Finalmente se generan los índices que definen los triángulos que forman la geometría. En cada iteración se crean dos triángulos para las paredes laterales y un triángulo adicional para la base y la tapa, utilizando los vértices centrales previamente definidos.

```
void CrearCilindro(int res, float R) {
    GLfloat dt = 2 * PI / res;
    vector<GLfloat> vertices;
    vector<unsigned int> indices;

    for (int n = 0; n <= res; n++) {
        float x = R * cos(n * dt);
        float z = R * sin(n * dt);
        vertices.push_back(x); vertices.push_back(-0.5f); vertices.push_back(z); // Abajo
        vertices.push_back(x); vertices.push_back(0.5f);  vertices.push_back(z); // Arriba
    }
    unsigned int sideCount = (res + 1) * 2;
    unsigned int bottomStart = sideCount;
    unsigned int topStart = bottomStart + (res + 1);
    for (int n = 0; n <= res; n++) {
        float x = R * cos(n * dt);
        float z = R * sin(n * dt);
        vertices.push_back(x); vertices.push_back(-0.5f); vertices.push_back(z);
    }
    for (int n = 0; n <= res; n++) {
        float x = R * cos(n * dt);
        float z = R * sin(n * dt);
        vertices.push_back(x); vertices.push_back(0.5f); vertices.push_back(z);
    }

    unsigned int bottomCenter = (unsigned int)(vertices.size() / 3);
    vertices.push_back(0.0f); vertices.push_back(-0.5f); vertices.push_back(0.0f);
    unsigned int topCenter = (unsigned int)(vertices.size() / 3);
    vertices.push_back(0.0f); vertices.push_back(0.5f); vertices.push_back(0.0f);

    for (int n = 0; n < res; n++) {
        unsigned int b0 = 2 * n, t0 = 2 * n + 1, b1 = 2 * (n + 1), t1 = 2 * (n + 1) + 1;
        indices.push_back(b0); indices.push_back(t0); indices.push_back(t1);
        indices.push_back(b0); indices.push_back(t1); indices.push_back(b1);
        indices.push_back(bottomCenter);
        indices.push_back(bottomStart + n + 1);
        indices.push_back(bottomStart + n);

        indices.push_back(topCenter);
        indices.push_back(topStart + n);
        indices.push_back(topStart + n + 1);
    }

    Mesh* cilindro = new Mesh();
    cilindro->CreateMeshGeometry(vertices, indices, vertices.size(), indices.size());
    meshList.push_back(cilindro);
}
```

Este cilindro se utiliza para representar los troncos de los árboles.

Cono

La función CrearCono(int res, float R) genera la geometría de un cono cerrado utilizando la misma idea de discretizar una circunferencia mediante las ecuaciones paramétricas.

Primero se agrega el vértice que representa la punta del cono. Después se generan los vértices correspondientes a la circunferencia de la base utilizando las ecuaciones paramétricas x=Rcos(θ) y z=Rsin(θ).

Posteriormente se agrega un vértice central en la base del cono, el cual permite cerrar la superficie inferior.

Finalmente se generan los índices que forman los triángulos del cono. Cada iteración del ciclo genera dos triángulos: uno que conecta la punta con dos puntos consecutivos de la base para formar la superficie lateral, y otro que conecta el centro de la base con esos mismos puntos para formar la base del cono.

```
void CrearCono(int res, float R) {
    GLfloat dt = 2 * PI / res;
    vector<GLfloat> vertices;
    vector<unsigned int> indices;

        vertices.push_back(0.0f); vertices.push_back(0.5f); vertices.push_back(0.0f);
        for (int n = 0; n <= res; n++) {
        vertices.push_back(R * cos(n * dt));
        vertices.push_back(-0.5f);
        vertices.push_back(R * sin(n * dt));
    }
    unsigned int center = (unsigned int)(vertices.size() / 3);
    vertices.push_back(0.0f); vertices.push_back(-0.5f); vertices.push_back(0.0f);
        for (int n = 0; n < res; n++) {
            indices.push_back(0);
        indices.push_back(1 + n);
        indices.push_back(2 + n);
        indices.push_back(center);
        indices.push_back(2 + n);
        indices.push_back(1 + n);
    }
        Mesh* cono = new Mesh();
    cono->CreateMeshGeometry(vertices, indices, vertices.size(), indices.size());
    meshList.push_back(cono);
}
```

Este cono se utiliza para representar la copa de los árboles.

Pirámide cuadrangular

La pirámide cuadrangular se utilizó para representar el techo de la casa.

Se generan:

- 4 caras laterales (4 triángulos)
- 2 triángulos para la base

```
void CrearPiramideCuadrangular() {
    vector<unsigned int> indices = { 0,4,3, 3,4,2, 2,4,1, 1,4,0, 0,2,1, 0,3,2 };
    vector<GLfloat> vertices = { 0.5f,-0.5f,0.5f, 0.5f,-0.5f,-0.5f, -0.5f,-0.5f,-0.5f, -0.5f,-0.5f,0.5f, 0.0f,0.5f,0.0f };
    Mesh* piramide = new Mesh();
    piramide->CreateMeshGeometry(vertices, indices, 15, 18);
    meshList.push_back(piramide); // meshList[3] [cite: 44]
}
```

Esfera

La esfera no se genera manualmente en el main, sino mediante la clase Sphere, que internamente crea vértices/índices a partir del radio y la resolución (slices/stacks). En el ciclo de render solo se aplica la transformación y se manda a dibujar.

```
Sphere sp = Sphere(1.0, 20, 20);

sp.init();
sp.load();

// ...
model = glm::translate(glm::mat4(1.0f), glm::vec3(...));
model = glm::scale(model, glm::vec3(...));
glUniformMatrix4fv(uniModel, 1, GL_FALSE, glm::value_ptr(model));
sp.render();
```

La escena final se construyó mediante la instanciación de las primitivas geométricas generadas previamente. Cada objeto se posicionó en el espacio tridimensional utilizando matrices de transformación, específicamente traslación y escalamiento mediante la librería **GLM**.

Ahora en el main tenemos que:

```
int main() {
    mainWindow = Window(800, 800);
    mainWindow.Initialise();
    glEnable(GL_DEPTH_TEST);
    CrearCubo(); CrearCilindro(250, 1.0f); CrearCono(25, 2.0f); CrearPiramideCuadrangular();
    Shader* s1 = new Shader(); s1->CreateFromFiles(vShader, fShader); shaderList.push_back(*s1);
    camera = Camera(glm::vec3(0.0f, 0.0f, 0.0f), glm::vec3(0.0f, 1.0f, 0.0f), -90.0f, 0.0f, 0.3f, 0.3f);
    glm::mat4 projection = glm::perspective(glm::radians(60.0f), mainWindow.getBufferWidth() / mainWindow.getBufferHeight(), 0.1f, 100.0f);
    sp.init(); sp.load();

    while (!mainWindow.getShouldClose()) {
        GLfloat now = glfwGetTime();
        deltaTime = now - lastTime;
        deltaTime += (now - lastTime) / limitFPS;
        lastTime = now;
        glfwPollEvents();
        camera.keyControl(mainWindow.getsKeys(), deltaTime);
        camera.mouseControl(mainWindow.getXChange(), mainWindow.getYChange());
        glClearColor(0.0f, 0.0f, 0.0f, 1.0f);
        glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
        shaderList[0].useShader();
        GLuint uniModel = shaderList[0].getModelLocation();
        GLuint uniProj = shaderList[0].getProjectLocation();
        GLuint uniView = shaderList[0].getViewLocation();
        GLuint uniCol = shaderList[0].getColorLocation();
        glUniformMatrix4fv(uniProj, 1, GL_FALSE, glm::value_ptr(projection));
        glUniformMatrix4fv(uniView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
        // --- RENDER ESCENA ---
        // PISO
        glm::mat4 model = glm::translate(glm::mat4(1.0f), glm::vec3(0.0f, -1.8f, -6.0f));
        model = glm::scale(model, glm::vec3(12.0f, 0.1f, 8.0f));
        glUniform3fv(uniCol, 1, glm::value_ptr(glm::vec3(0.8f, 0.8f, 0.8f)));
        glUniformMatrix4fv(uniModel, 1, GL_FALSE, glm::value_ptr(model));
        meshList[0]->RenderMesh();
        // CUERPO CASA
        model = glm::translate(glm::mat4(1.0f), glm::vec3(0.0f, 0.2f, -6.0f));
        model = glm::scale(model, glm::vec3(2.5f, 4.0f, 2.5f));
        glUniform3fv(uniCol, 1, glm::value_ptr(glm::vec3(1.0f, 0.0f, 0.0f)));
        glUniformMatrix4fv(uniModel, 1, GL_FALSE, glm::value_ptr(model));
        meshList[0]->RenderMesh();
```

```
        // TECHO
        model = glm::translate(glm::mat4(1.0f), glm::vec3(0.0f, 3.0f, -6.0f));
        model = glm::scale(model, glm::vec3(3.5f, 2.5f, 3.5f));
        glUniform3fv(uniCol, 1, glm::value_ptr(glm::vec3(0.0f, 0.0f, 1.0f)));
        glUniformMatrix4fv(uniModel, 1, GL_FALSE, glm::value_ptr(model));
        meshList[3]->RenderMeshGeometry();
        // PUERTA
        model = glm::translate(glm::mat4(1.0f), glm::vec3(0.0f, -1.3f, -4.74f));
        model = glm::scale(model, glm::vec3(0.8f, 1.0f, 0.02f));
        glUniform3fv(uniCol, 1, glm::value_ptr(glm::vec3(0.0f, 1.0f, 0.0f)));
        glUniformMatrix4fv(uniModel, 1, GL_FALSE, glm::value_ptr(model));
        meshList[0]->RenderMesh();
        // VENTANAS FRONTALES
        glUniform3fv(uniCol, 1, glm::value_ptr(glm::vec3(0.0f, 1.0f, 0.0f))); // Verde
        for (float x : {-0.65f, 0.65f}) {
            model = glm::translate(glm::mat4(1.0f), glm::vec3(x, 0.9f, -4.74f));
            model = glm::scale(model, glm::vec3(0.7f, 0.8f, 0.02f));
            glUniformMatrix4fv(uniModel, 1, GL_FALSE, glm::value_ptr(model));
            meshList[0]->RenderMesh();
        }
        // VENTANAS LATERALES
        for (float x : {-1.26f, 1.26f}) {
            for (float z : {-6.5f, -5.5f}) {
                model = glm::translate(glm::mat4(1.0f), glm::vec3(x, 0.9f, z));
                model = glm::scale(model, glm::vec3(0.02f, 0.8f, 0.8f));
                glUniformMatrix4fv(uniModel, 1, GL_FALSE, glm::value_ptr(model));
                meshList[0]->RenderMesh();
            }
        }
        // VENTANA CIRCULAR TRASERA
        model = glm::translate(glm::mat4(1.0f), glm::vec3(0.0f, -0.2f, -7.26f));
        model = glm::scale(model, glm::vec3(0.8f, 0.8f, 0.05f));
        glUniform3fv(uniCol, 1, glm::value_ptr(glm::vec3(0.0f, 0.0f, 1.0f)));
        glUniformMatrix4fv(uniModel, 1, GL_FALSE, glm::value_ptr(model));
        sp.render();
        // ÁRBOLES
        for (float x : {-3.2f, 3.2f}) {
            model = glm::translate(glm::mat4(1.0f), glm::vec3(x, -1.2f, -6.0f));
            model = glm::scale(model, glm::vec3(0.5f, 1.2f, 0.5f));
            glUniform3fv(uniCol, 1, glm::value_ptr(glm::vec3(0.35f, 0.25f, 0.1f)));
            glUniformMatrix4fv(uniModel, 1, GL_FALSE, glm::value_ptr(model));
            meshList[1]->RenderMeshGeometry();
            model = glm::translate(glm::mat4(1.0f), glm::vec3(x, 0.3f, -6.0f));
            model = glm::scale(model, glm::vec3(0.4f, 2.0f, 0.4f));
            glUniform3fv(uniCol, 1, glm::value_ptr(glm::vec3(0.0f, 0.6f, 0.0f)));
            glUniformMatrix4fv(uniModel, 1, GL_FALSE, glm::value_ptr(model));
            meshList[2]->RenderMeshGeometry();
        }
        mainWindow.swapBuffers();
    }
    return 0;
}
```

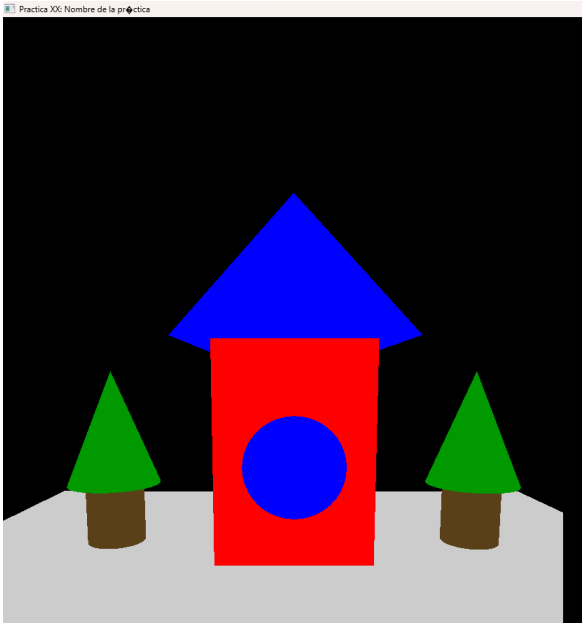
En la función main se inicializa todo lo necesario para poder mostrar la escena en pantalla. Primero se crea la ventana donde se va a dibujar la escena y se activan las opciones necesarias para que los objetos se vean correctamente en 3D. Después se generan las figuras geométricas que se van a utilizar, como el cubo, el cilindro, el cono y la pirámide, las cuales se almacenan para poder reutilizarlas en diferentes partes de la escena. También se inicializan el shader, la cámara y la proyección que permite observar los objetos con profundidad.

Posteriormente se ejecuta un ciclo que se mantiene activo mientras la ventana esté abierta. Dentro de este ciclo se actualizan los movimientos de la cámara con el teclado y el mouse, se limpia la pantalla y se dibujan todos los elementos que forman la escena. Para construir la escena se reutilizan las primitivas geométricas y se posicionan en distintos lugares mediante transformaciones, lo que permite formar el piso, la casa, las ventanas, la puerta y los árboles. Este proceso se repite continuamente hasta que el usuario cierra la ventana.

Finalmente obteniendo la imagen esperada

Practica XX: Nombre de la pr□ctica

Practica XX: Nombre de la pr□ctica


Practica XX: Nombre de la pr□ctica

Practica XX: Nombre de la pr□ctica

Problemas presentados.

Durante el desarrollo y ejecución del programa se presentaron algunos problemas relacionados con la generación y visualización de las primitivas geométricas. Uno de los principales inconvenientes fue que algunas caras de las figuras no se dibujaban correctamente, en particular en la pirámide utilizada para el techo de la casa y en los conos que forman la copa de los árboles. Esto ocurría porque el modo de dibujo utilizado para renderizar las geometrías no correspondía con la forma en la que estaban definidos los índices de los triángulos.

Para solucionar este problema se modificó el modo de dibujo en el archivo Mesh.cpp, cambiando la instrucción que utilizaba GL_TRIANGLE_FAN por GL_TRIANGLES. De esta manera, cada grupo de tres índices se interpreta como un triángulo independiente, lo que permitió que las caras de las figuras se generaran correctamente.

Otro problema que se presentó fue que algunas figuras, como el cilindro y el cono, inicialmente se dibujaban sin base o tapa, lo que hacía que se vieran incompletas. Para resolverlo fue necesario agregar vértices adicionales correspondientes al centro de las bases y generar los triángulos necesarios para cerrar estas superficies.

Finalmente, también se ajustaron algunos valores de escala y posición de las figuras dentro de la escena, ya que al principio algunos objetos se veían desproporcionados o mal colocados. Una vez realizados estos ajustes, la escena pudo visualizarse correctamente con todos sus elementos.

Conclusión

a) Los ejercicios de la clase: Complejidad, explicación

En esta práctica se trabajó con el modelado de primitivas geométricas y la construcción de una escena tridimensional utilizando OpenGL. Aunque al inicio las figuras básicas como el cubo o la pirámide pueden parecer sencillas, al implementarlas se vuelve más claro que cada objeto en 3D debe construirse a partir de vértices e índices que forman triángulos. Esto hace que la complejidad aumente cuando se desea generar figuras más elaboradas como cilindros o conos, ya que es necesario calcular los puntos de una circunferencia mediante funciones trigonométricas y conectar correctamente los vértices para formar las caras.

Además, la práctica no solo consistió en generar las figuras, sino también en reutilizarlas para construir una escena completa. Esto implicó aplicar transformaciones de posición y escala para colocar los objetos en diferentes

partes del espacio y así formar la casa, el piso, las ventanas y los árboles. En general, el ejercicio permite entender mejor cómo se construyen objetos más complejos a partir de primitivas simples y cómo se organizan dentro de una escena tridimensional.

b) Comentarios generales

En general la práctica fue interesante porque permite ver de manera más clara cómo se construyen las figuras en gráficos por computadora. Sin embargo, en algunos momentos las explicaciones pueden resultar un poco rápidas, especialmente en la parte donde se generan los vértices e índices de las figuras geométricas.

También podría ser útil mostrar más ejemplos paso a paso de cómo se generan otras primitivas geométricas, para que pueda practicar y entender mejor el proceso. En general, la práctica ayuda bastante a entender el uso de transformaciones y la construcción de escenas en OpenGL, pero ir un poco más lento en algunas explicaciones podría facilitar el aprendizaje.